Homework 8: Inheritance and Exceptions

Total Points: 100 (Correctness and Style) + 3 EC (Checkpoint)

Due Dates (SD time):

- **Entire Assignment: Tuesday, May 27th, 11:59pm**
- **Checkpoint (read below): Sunday, May 25th, 11:59pm**

Starter Files

Download hw08.zip Inside the archive, you will find starter files for the questions of this homework. You cannot import anything to solve the problems.

IMPORTANT: Coding and Docstring Style

This is a reminder that your code style will be graded. Here are a few useful links:

Link to the requirements (I moved it to the separate page from now on).

Style Guide on Course Website

Style Guide Examples

Testing

At any point of the homework, use one of the following commands to test your work:

```
>>> python3 -m doctest hw08.py
>>> py -m doctest hw08.py
>>> python -m doctest hw08.py
```

Checkpoint Submission

Due Date: Sunday, May 25th, 11:59pm (SD time)

You can earn **3 points extra credit** by submitting the checkpoint by the due date above. In the checkpoint submission, you should complete:

- **Question 1**

and submit the hw08.py file to gradescope.

Checkpoint submission is graded by completion, which means you can get full points if your code can pass some simple sanity check (no tests against edge cases). Note that in your final submission, you should still submit these questions, and you may modify your implementation if you noticed any errors.

Required Questions

1. **DO NOT IMPORT ANY PACKAGES.**
2. Please add your own doctests (**at least three**) as the given doctests are not sufficient. **You will be graded on the doctests**.
   - **No doctests are needed for the entirety of Question 2.**

   For Question 1 Doctests:
   a. You do not need to add doctests for each constructor/getter/setter. Instead, create at least 3 new objects of each class and subclass in order to test constructors, getters, setters as a whole.
   b. For every non-getter and non-setter method, add at least 3 new doctests.
   c. When adding doctests, add them to the bottom of the q1_doctests() docstring. There is a comment indicating where you should add your doctests. This will make it easier for graders to see your new doctests so you don't get marked off.
3. Docstrings are provided for Question 1.

Question 1:

Harry Potter is a theme that attracts many people around the world, so your team decided to develop a game based on it.

In this question, you will implement classes that abstract different Broom types in the game Broom Chase. Doctests for this question are in the method *q1_doctests()*. Please add more doctests as specified here.

Part 1: Class FlyingBroom

The class **FlyingBroom** represents a general broomstick in the wizarding world.

In the constructor (`__init__`), you must initialize four instance attributes (properties) with specific default values. No parameters will be passed in:

- **speed** (*non-negative integer*): Represents the current flying speed of the broom. This takes a default value of 50.
- **size** (*positive integer*): Represents the physical size of the broom. This takes a default value of 5.
- **magic_power** (*non-negative integer*): Represents the number of magical ws remaining for this broom. This is initialized to 3.
- **lives** (*non-negative integer*): Represents the number of lives a wizard has while flying this broom. This is initialized to 3.

In addition to the instance attributes, you should also define the following functions:

- **boost(self, charm_power)** this function boosts the speed of the broom by using a magical charm. The charm_power is **applied as long as the broom still has some magic power remaining**. The speed boost is calculated using the formula:
  `new_speed` = int($\sqrt{(oldSpeed + charm_power)^2 + (oldSpeed - charm_power)^2}$).
  - If this boost increases the broom's high score to at least double its previous score, the wizard gains an extra life.
  - After applying the boost, the `magic_power` attribute is decreased by one. If the boost is successfully applied (i.e., the broom had magic

power to perform the boost), the function returns `True`. If there is no remaining magic power to perform the boost, it returns `False`.

- Setter Methods:
  - **`set_speed`**`(self, new_speed)`: Assigns the given value to the `speed` attribute.
  - **`set_lives`**`(self, gains=True)`: Takes in a boolean parameter. If `gains` is `True`, increment the `lives` attribute by `1`; otherwise, decrement it by `1`.
  - **`set_size`**`(self, new_size)`: Assigns the given non-negative value to the `size` attribute.

- **duel(self, other_broom):** Takes in another broom instance and determines if a duel can occur.

  A duel is possible if the current broom's size is larger than the other broom's size. If so, the following happens:

  - The other_broom's speed is reduced by 50, and the current broom's speed is increased by 50.
  - If the `other_broom's` speed becomes non-positive:
    - The wizard riding the `other_broom` loses a life, and their speed resets to 50.
    - The current broom's size increases by 1.

- If the other broom is larger, the same rules apply but reversed (i.e., perform the same steps as above but switch `other_broom` and `self`).
- If both brooms are the same size, nothing happens.

Return `True` only if the current broom successfully performs a duel; otherwise, return `False`.

- **high_score(self)** returns the high score using the following formula:
  score = speed * 100 + lives * 500

<u>Part 2: Class NormalBroom</u>

This is a subclass of the **FlyingBroom** class. It inherits all the properties and methods from **FlyingBroom**, with one key difference in the `duel` method:

- If the `other_broom` object is an instance of **CursedBroom**, the current **NormalBroom** loses one life, and its speed resets to 30. Additionally, the **CursedBroom** object gains a size and has its speed increased by 50.
- In this scenario, the attack is considered unsuccessful, and the function should return `False`.
- If the `other_broom` is **not** an instance of **CursedBroom**, the functionality of `duel` is the same as in the parent class. Use `super()` to call the parent class method when needed.

Part 3: Class CursedBroom

This is a subclass of the **FlyingBroom** class. It inherits all attributes and methods from **FlyingBroom**, with the following differences:

1. **Initialization:**
    - A **CursedBroom** has different default values:
        - `speed = 70`
        - `size = 7`
        - `magic_power = 5`
        - `lives = 5`
2. **High Score Calculation:**
    - The high score for a **CursedBroom** is calculated differently: `score = speed * 200 + lives * 300 + 250`.

Overwrite the parent class's method as needed.

Exceptions

Question 2

In this question, you need to fix the implementation of three functions so that they no longer crash. The provided implementation will not work for certain values due to exceptions, thus you need to add a try-except block in each function to deal with this. You should run the doctests to see which exceptions will be thrown. **No doctests are needed.**

For this question, all you need to do is add a try-except block to handle the potential exception. You should not modify the actual implementation of the given code, and you are not required to add input validations.

Part 1

This function is supposed to divide each element in *lst1* by each element in *lst2*, and append each result to an output list. However, our current implementation results in a crash for certain values. If we catch the exception, we should just move on to the next iteration of the loop instead of terminating the entire loop.

```
def fix_1(lst1, lst2):
    """
    >>> fix_1([1, 2, 3], [0, 1])
    [1.0, 2.0, 3.0]
    >>> fix_1([], [])
    []
    >>> fix_1([10, 20, 30], [0, 10, 10, 0])
    [1.0, 2.0, 3.0, 1.0, 2.0, 3.0]
    """
    out = []
    for div in lst2:
        for num in lst1:
            out.append(num / div) # add try-catch block
    return out
```

Part 2

This function is supposed to open each filepath in **filepaths*.

If we are able to open the file, we should **print** a string '*{filepath}* opened'. If we are not able to open the file, we should **print** '*{filepath}* not found'. Note that *{filepath}* should be the given file path string, not the actual string '{filepath}'.

```
def fix_2(*filepaths):
    """
    >>> fix_2('files/a.txt', 'files/b.txt', 'files/c.txt')
    files/a.txt opened
```

```
    files/b.txt not found
    files/c.txt not found

    >>> fix_2('docs.txt')
    docs.txt not found
    """
    for filepath in filepaths:
        cur_file = open(filepath, "r") # add try-catch block
        cur_file.close()
```

Part 3

This function is supposed to add each element with its following element in the list and return all of the summed values in a list.

In the current implementation, there are two potential errors that may arise. If they are thrown, you should catch them and print their types (you can use the type() function). Otherwise, you would just append lst[i] + lst[i+1] to the output list.

```
def fix_3(lst):
    """
    >>> fix_3([1, '1', 2, None])
    <class 'TypeError'>
    <class 'TypeError'>
    <class 'TypeError'>
    <class 'IndexError'>
    []

    >>> fix_3([1, 2, 3, 4])
    <class 'IndexError'>
    [3, 5, 7]

    >>> fix_3([])
    []
    """
    sum_of_pairs = []
    for i, _ in enumerate(lst):
        sum_of_pairs.append(lst[i] + lst[i + 1]) # add try-catch block
```

```
    return sum_of_pairs
```

Question 3

For this question we will be checking the correctness of input1 and input2 using **exceptions** and **NOT assertions**. If a check fails, you will throw an *exception*. You should make sure you are doing the following checks **in order** to prevent issues with the doctests expecting the wrong exception.
Checks (in this order):

1. `input1` should be a list
2. All of the values in `input1` should be numeric. It is ok if `input1` is empty
    a. If there are multiple non-numeric values, you only need to throw an exception for the first non-numeric value encountered
3. `input2` should be numeric
4. `input2` should be contained in `input1`

If all of the checks pass, you should **return** the string 'Input validated'

Refer to the doctests to see which exceptions are thrown for each of the checks and how the corresponding messages should be formatted.

```
def check_inputs(input1, input2):
    """
    >>> check_inputs([1, 2.0, 3.0, 4], 4)
    'Input validated'
    >>> check_inputs([], 1)
    Traceback (most recent call last):
    ...
    TypeError: input2 not in input1
    >>> check_inputs(1, 1)
    Traceback (most recent call last):
    ...
    TypeError: input1 is not the correct type
    >>> check_inputs([1, 2, 'hi'], 4)
    Traceback (most recent call last):
    ...
    TypeError: The element at index 2 is not numeric
    >>> check_inputs([1.0, 2.0, 3.0], 'hello')
    Traceback (most recent call last):
    ...
    TypeError: input2 is not the correct type
```

```
    """
    return …
```

Question 4

For this question we will be checking the correctness of a given `filepath` and its corresponding file using **exceptions** and **NOT assertions**. If a check fails, you will throw an exception. You should make sure you are doing the following checks **in order** to prevent issues with the doctests expecting the wrong exception.

Checks (in this order):

1. `filepath` should be a string
2. `filepath` should be a valid `filepath` (i.e. we can open the file)
3. The file at `filepath` should not be empty (i.e. there should be >= 1 words in the entire file)

If all of the checks pass, you should return the number of words in the file.

Refer to the doctests to see which exceptions are thrown for each of the checks and how the corresponding messages should be formatted.

```
def load_file(filepath):
    """
    >>> load_file(1)
    Traceback (most recent call last):
    ...
    TypeError: filepath is not a string

    >>> load_file('files/ten_words.txt')
    10

    >>> load_file('files/empty.txt')
    Traceback (most recent call last):
    ...
    ValueError: File is empty

    >>> load_file('files/nonexistant.txt')
    Traceback (most recent call last):
    ...
    FileNotFoundError: files/nonexistant.txt does not exist
```

```
    """
    return ...
```